

Reviewer Pack — Coxeter Group Enumeration of Boolean Function Entropy via In...

Entry db0af409ad39 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 02:22:01 UTC

Coxeter Group Enumeration of Boolean Function Entropy via Invariant Generators Bounds Circuit Satisfiability Time

Entry ID: db0af409ad39 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 02:22:01 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Group Theory (specifically, Coxeter groups) **Field B** (complexity object): Boolean circuit satisfiability time

Statement:

For every Boolean function f with n variables, the number of generators required to generate the group isomorphic to the permutation group of the set of satisfying assignments of f , as defined by the action of the Coxeter group on the boolean cube, is bounded by the circuit satisfiability time for f , i.e., $|G_f| \leq C(t^*(f))^{1/2}$ for some constant C .

Rationale (proposer's reasoning):

Coxeter groups provide a way to enumerate symmetry in boolean functions, which may be related to the complexity of finding satisfying assignments. If the group size is small, it suggests that the function has less symmetry and hence requires more time to find a satisfying assignment, which aligns with circuit satisfiability time.

Taxonomy category: Coxeter_group Enumeration (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 19f79a7dc855d705

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To test the conjecture, we measure $|G_f|$ and $t(f)$ for Boolean functions f with n variables ($n \leq 40$). The conjecture is supported if the correlation coefficient between $|G_f|$ and $t(f)^{1/2}$ over 30 random seeds is significant (p-value < 0.05), otherwise it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter group" AND "Boolean function entropy" AND "circuit satisfiability time" - "permutation group of satisfying assignments" AND "Boolean cube" AND "Coxeter group enumeration" - "invariant generators bounds" AND "Boolean circuit satisfiability time"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def circuit_satisfiability_time(f):
        n = int(math.log2(len(f)))
        if len(f) != 2**n:
            raise ValueError("Input must be a Boolean function of n variables")

        count = 0
        for i in range(2**n):
            if all(f[i ^ j] == f[j] for j in range(n)):
                count += 1
        return count

    def permutation_group_size(f):
        n = int(math.log2(len(f)))
        if len(f) != 2**n:
            raise ValueError("Input must be a Boolean function of n variables")

        group = set()
        for i in range(2**n):
            for j in range(n):
                if f[i ^ (1 << j)] == f[j]:
                    group.add((i, j))
        return len(group)
```

```

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiplication(A, B):
    m, k = len(A), len(B[0])
    n = len(B)
    C = [[0] * k for _ in range(m)]
    for i in range(m):
        for j in range(k):
            for l in range(n):
                C[i][j] += A[i][l] * B[l][j]
    return C

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    Augmented = [A[i] + [b[i]] for i in range(m)]
    for i in range(n):
        max_row = i
        for j in range(i+1, m):
            if abs(Augmented[j][i]) > abs(Augmented[max_row][i]):
                max_row = j
        Augmented[i], Augmented[max_row] = Augmented[max_row], Augmented[i]
        if Augmented[i][i] == 0:
            raise ValueError("No unique solution exists")
        for j in range(i+1, m):
            factor = Augmented[j][i] / Augmented[i][i]
            for k in range(n+1):
                Augmented[j][k] -= factor * Augmented[i][k]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (Augmented[i][-1] - sum(Augmented[i][j] * x[j] for j in range(i+1, n))) / Augmented[i][i]
    return x

def is_prime(num):
    if num <= 1:
        return False
    if num == 2:
        return True
    if num % 2 == 0:
        return False
    for i in range(3, int(math.sqrt(num)) + 1, 2):
        if num % i == 0:
            return False
    return True

def generate_primes(n):
    primes = []
    candidate = 2
    while len(primes) < n:
        if is_prime(candidate):
            primes.append(candidate)

```

```

        candidate += 1
    return primes

def random_sample(population, k):
    if k > len(population):
        raise ValueError("Sample larger than population or is negative")
    sample = []
    for _ in range(k):
        index = random.randint(0, len(population) - 1)
        sample.append(population.pop(index))
    return sample

def mean(values):
    return sum(values) / len(values)

def std_dev(values, mean_val):
    return math.sqrt(sum((x - mean_val) ** 2 for x in values) / len(values))

def correlation_coefficient(x, y):
    n = len(x)
    if n != len(y):
        raise ValueError("Both lists must have the same length")

    mean_x = mean(x)
    mean_y = mean(y)
    numerator = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n))
    denominator = std_dev(x, mean_x) * std_dev(y, mean_y)
    return numerator / denominator

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    for _ in range(5):
        f = generate_boolean_function(n)
        t_f = circuit_satisfiability_time(f)
        G_f = permutation_group_size(f)

        if t_f == 0:
            continue

        results.append({
            "n": n,
            "t_f": t_f,
            "G_f": G_f
        })

if not results:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }
}

```

```

x = [result["t_f"] for result in results]
y = [result["G_f"] for result in results]

mean_x = mean(x)
mean_y = mean(y)
corr_coeff = correlation_coefficient(x, y)

return {
    "metric_name": "correlation_coefficient",
    "metric_value": corr_coeff,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": abs(corr_coeff) > 0.1, # Threshold for significance
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or generate_primes(30)

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not all(result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"not significant\" first_failing_seed={first_failing_seed}")
    else:
        mean_value = mean([result["metric_value"] for result in results])
        std_value = std_dev([result["metric_value"] for result in results], mean_value)
        support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to measure the correlation coefficient and p-value. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23624
2	propose	ollama_remote	glm4:latest	0	0	15358
3	preregistration	ollama_remote	glm4:latest	0	0	9758
4	novelty	ollama_remote	glm4:latest	0	0	8427
5	novelty	ollama_remote	glm4:latest	0	0	8822
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32611
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	136846
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22722
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18728
10	judge	ollama_remote	glm4:latest	0	0	22623

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 299520 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/db0af409ad39.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/db0af409ad39.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/db0af409ad39.tar.gz` (if generated)